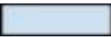
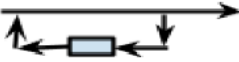




Practica 2

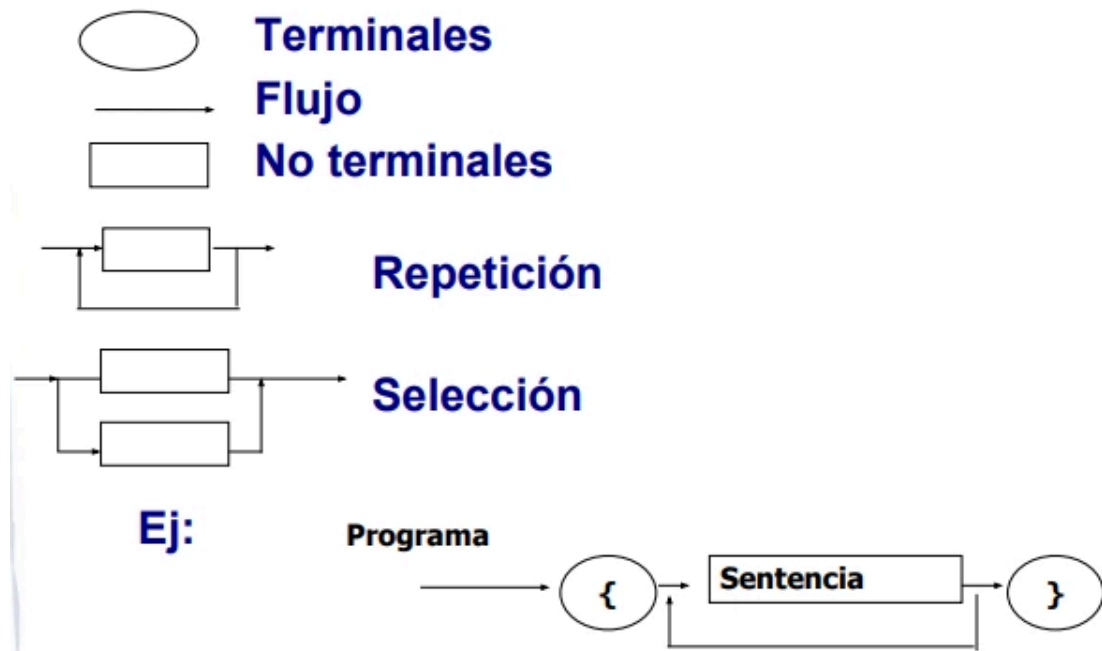
Practica2_2026.pdf

Ejercicio 1: Complete el siguiente cuadro:

Meta símbolos utilizados por		Símbolo utilizado en Diagramas sintácticos	Significado
BNF	EBNF		
palabra terminal	palabra terminal		Definición de un elemento terminal
< >	< >	rectángulo 	Definición de un elemento no terminal
::=	::=	diagrama con rectángulos, óvalos y flechas	Es un metasímbolo y define una producción / regla
	()	flecha que se divide en dos o más caminos	Selección de una alternativa
< p > < p1 >	{ }	flecha que vuelve a la condición	Repetición
	*		Repetición de 0 o más veces
	+		Repetición de 1 o más veces
	[]		Opcional, está presente o no lo está

-Diagramas sacados del apunte:

Diagramas sintácticos (CONWAY):



Ejercicio 2: ¿Cuál es la importancia de la sintaxis para un lenguaje? ¿Cuáles son sus elementos?

-La sintaxis son un conjunto de reglas que definen como componer letras, digitos y otros caracteres para formar los programas. Permite escribir programas bien escritos, donde la comunicacion es clara y ver el resultado al ejecutarlo

-Sus elementos son:

-Alfabeto/Conjunto de caracteres:

-Con que conjunto de caracteres se trabaja

-Secuencia de bits que compone cada carácter determina la implementación.

-Identificadores → Cadena de letras y digitos que deben comenzar con una letra. Si se restringe pierde legibilidad

-Operadores → Suma, resta, %. La mayoría usa + - . En otros operadores no hay tanta uniformidad

-Comentarios → Programas mas legibles

-Palabra clave/Reservada:

-Palabra clave → Tienen un significado dentro de un contexto

-Palabra reservada → No pueden ser utilizadas por el programador como identificador de otra entidad (ejemplo usar procedure como identificador o variable en Pascal)

Ejercicio 3: ¿Explique a qué se denomina regla lexicográfica y regla sintáctica?

-Reglas lexicas → Conjunto de reglas para formar las "word" a partir de caracteres del alfabeto. Diferencias entre mayusculas y minusculas.

Diferencias entre simbolo de distinto (<> , !=)

-Reglas sintacticas → Se definen para formas las expresiones y sentencias.

El if en C no lleva "then", pero en Pascal si

Ejercicio 4: ¿En la definición de un lenguaje, a qué se llama palabra reservadas? ¿A qué son equivalentes en la definición de una gramática? De un ejemplo de palabra reservada en el lenguaje que más conoce. (Ada,C,Ruby,Python,..)

-Palabra reservada → No pueden ser utilizadas por el programador como identificador de otra entidad. Tienen un significado especial por lo que esta restringido el uso de las mismas para nombrar variables o funciones

-Un ejemplo de una palabra reservada en python es **False**, o en Pascal **procedure**

Ejercicio 5: Dada la siguiente gramática escrita en BNF:

G= (N, T, S, P)

N = {<numero_entero>, <digito> }

T = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}

S = <numero_entero>

P = {

<numero_entero>::=<digito><numero_entero> | <numero_entero>

<digito> | <digito>

<digito> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

}

a- Identifique las componentes de la misma

-Los componentes son:

-G → Es la gramática, son reglas finitas que definen un conjunto infinito de posibles sentencias válidas en el lenguaje y se escribe con una 4-tupla (N,T,S,P)

-N → Conjunto de símbolos no terminales (<numero_entero> , <dígito>)

-T → Conjunto de símbolos terminales (0,1,2,3,4,5,6,7,8,9)

-S → Símbolo distinguido de la gramática (<numero_entero>)

-P → Es el conjunto de producciones de la gramática:

```
<numero_entero> ::= <dígito> <numero_entero> |  
<numero_entero> <dígito> | <dígito>  
<dígito> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
```

b- Indique por qué es ambigua y corrijala

-Es ambigua porque la sentencia puede derivarse de más de una forma, como se puede ver en la definición del <numero_entero> utiliza recursividad por izquierda y por derecha al mismo tiempo. Se tiene que usar solo una

Corrección:

G = (N, T, S, P)

N = { <numero_entero>, <dígito> }

T = { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 }

S = <numero_entero>

```
P = {  
    <numero_entero> ::= <dígito> | <dígito> <numero_  
entero>  
    <dígito> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8  
    | 9  
}
```

Ejercicio 6: Defina en BNF (Gramática de contexto libre desarrollada por Backus- Naur) la gramática para la definición de una palabra cualquiera

```
G = (N,T,S,P)

N = {<palabra>, <minus> , <mayus>, <letra>}

T = {A...Z , a...z}

S = <palabra>

P = {

<palabra> ::= <letra> | <letra> <palabra>

<letra> ::= <minus> | <mayus>

<minus> ::= a | b | c | ... | z

<mayus> ::= A | B | C | ... | Z

}
```

Ejercicio 7: Defina en EBNF la gramática para la definición de números reales. Inténtelo desarrollar para BNF y explique las diferencias con la utilización de la gramática EBNF.

```
EBNF:

G = (N,T,S,P)

N = {<real>, <digito>}

T = {0,1,2,3,4,5,6,7,8,9, "+", "-", ",", "."}

S = <real>

P = {
```

```
<real> ::= [-] {<digito>} + [ , {<digito>} * ]
```

```
<digito> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
```

```
}
```

BNF:

$G = (N, T, S, P)$

$N = \{<real>, <int>, <digito>\}$

$T = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, "+", "-", ",", "\}$

$S = <real>$

$P = \{$

```
<real> ::= <int> "," <int> | - <int> "," <int> | + <int>  
"," <int> |  
<int> | +<int> | - <int>
```

```
<int> ::= <digito> | <digito> <int>
```

```
<digito> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
```

```
}
```

-Para las posibles combinaciones BNF es mucho mas engorrosa y usa mas simbolos que EBNF

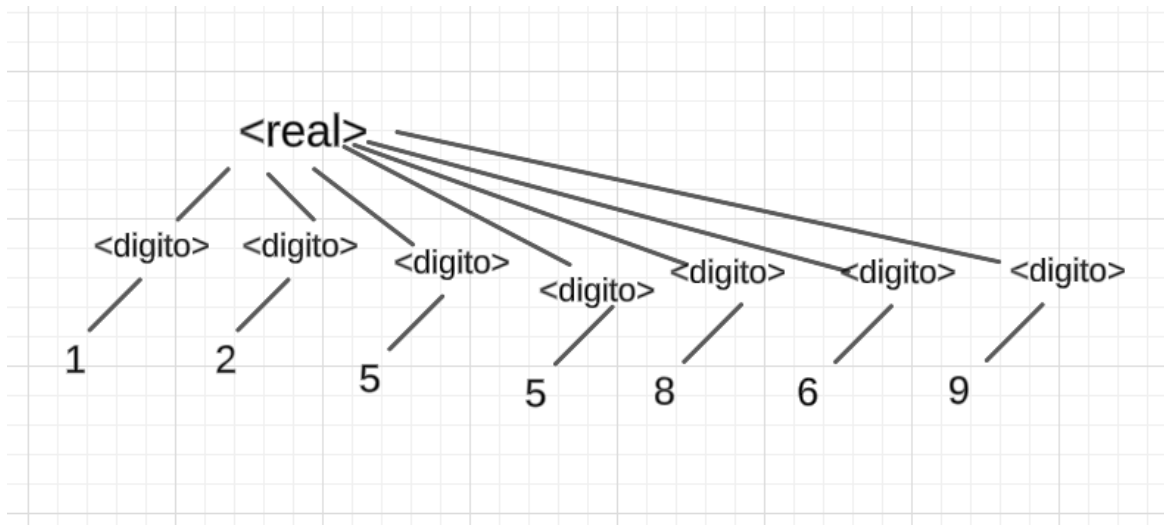
Ejercicio 8: Utilizando la gramática que desarrolló en los puntos 6 y 7, escriba el árbol sintáctico

de:

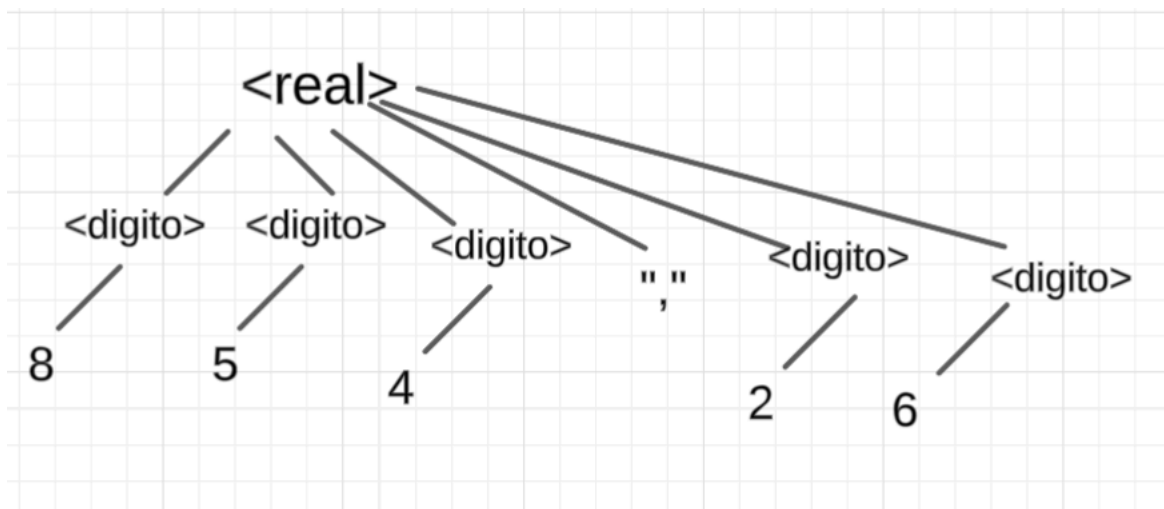
a. Conceptos



c. 1255869



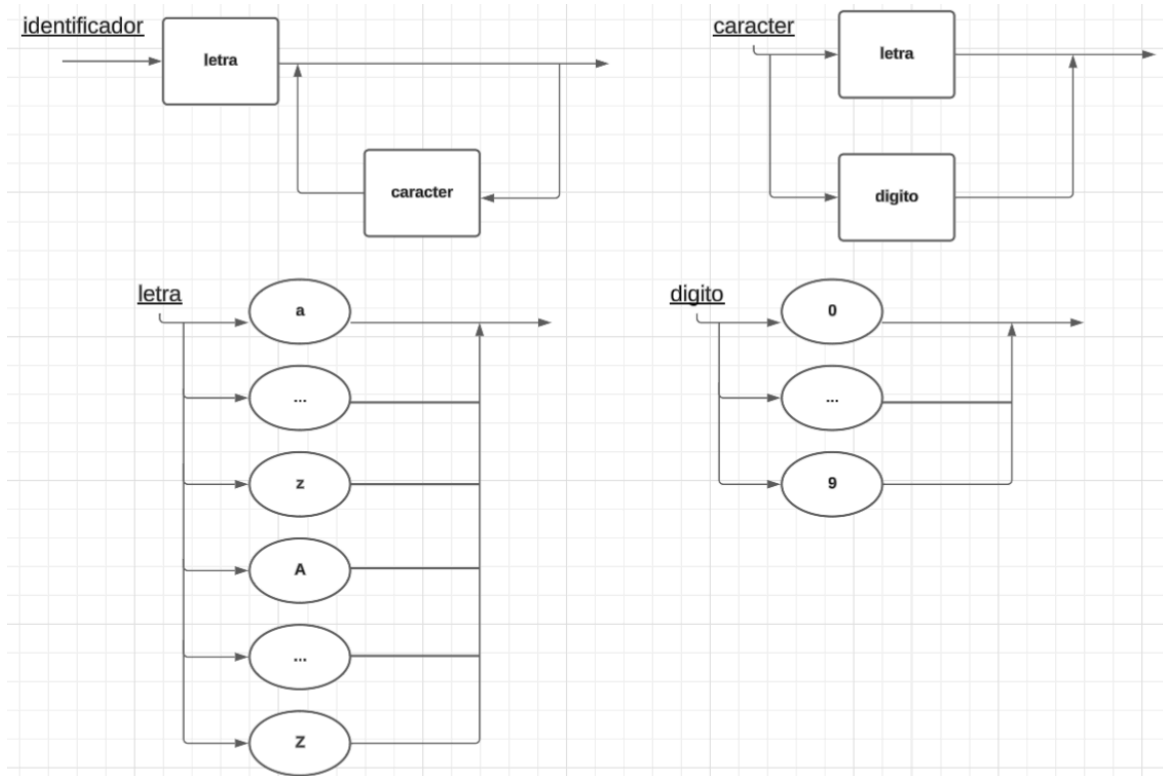
d. 854,26



e. Conceptos de lenguajes

-No se puede, la gramatica no contempla el " " como simbolo por lo que no se pueden representar palabras separadas por espacio. Se deberia agregar " " a la gramatica e implementarlo

Ejercicio 9: Defina utilizando diagramas sintácticos la gramática para la definición de un identificador de un lenguaje de programación. Tenga presente como regla que un identificador no puede comenzar con números.



Ejercicio 10:

a) Defina con EBNF la gramática para una expresión numérica, dónde intervienen variables y números. Considerar los operadores +, -, * y / sin orden de prioridad. No considerar el uso de paréntesis.

$G = (N, T, S, P)$

$N = \{ \langle \text{expr} \rangle, \langle \text{variable} \rangle, \langle \text{numero} \rangle, \langle \text{operador} \rangle, \langle \text{digito} \rangle, \langle \text{letra} \rangle \}$

$T = \{ a \dots z, A \dots Z, 0 \dots 9, "+", "-", "*", "/" \}$

$S = \langle \text{expr} \rangle$

$P = \{$

$\langle \text{expr} \rangle ::= (\langle \text{variable} \rangle \mid \langle \text{numero} \rangle) \{ \langle \text{operador} \rangle \{ (\langle \text{variable} \rangle \mid \langle \text{numero} \rangle) \}^+$

$\langle \text{numero} \rangle ::= [(+ \mid -)] \{ \langle \text{digito} \rangle \}^+$

```

<variable> ::= <letra> { ( <letra> | <digito> ) } *

<digito> ::= ( 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 )

<letra> ::= ( a | ... | z | A | ... | Z )

<operador> ::= ( + | - | * | / )

}

```

b) A la gramática definida en el ejercicio anterior agregarle prioridad de operadores.

```

G = (N,T,S,P)

N = {<expr>, <termino>, <letra>, <digito>, <elem>, <id>,
<numero>}

T = { a ... z, A ... Z, 0 ... 9, "+", "-", "*", "/" }

S = <expr>

P = {

<expr> ::= <termino> { ( + | - ) <termino> } *

<termino> ::= <elem> { ( * | / ) <elem> } *

<elem> ::= ( <id> | <numero> )

<numero> ::= <digito> { <digito> } *

<id> ::= <letra> { ( <letra> | <digito> ) } *

<letra> ::= ( a | ... | z | A | ... | Z )

<digito> ::= ( 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 )

```

}

c) Describa con sus palabras los pasos y decisiones que tomó para agregarle prioridad de operadores al ejercicio anterior.

-Al agregar <termino> y <elem> se establece una prioridad ya que para armar una expresion, es decir, <termino> + o - <termino>, primero necesitas armar un termino, ese termino va a detectar el "*" y el "/" a nivel bajo y se van a agrupar en <termino>. Entonces cuando se termina de armar el termino te quedan agrupadas las operaciones con mayor prioridad y suben a <expr> como <termino> juntas, y esos grupos van a estar separados por "+" o "-" en <expr>

Ejercicio 11: La siguiente gramática intenta describir sintácticamente la sentencia for de ADA, indique cuál/cuáles son los errores justificando la respuesta.

N= {<sentencia_for>, <bloque>, <variable>, <letra>, <cadena>, <digito>, <otro>, <operacion>, <llamada_a_funcion>, <numero>, <sentencia> }
P= {

<sentencia_for>::= for (i= IN 1..10) loop <bloque> end loop;
<variable>::= <letra> | <cadena>
<cadena>::= { (<letra> | <digito> | <otro>) }+
<letra>::= (a | .. | z | A | .. | Z)
<digito>::= (1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 0)
<bloque>::= <sentencia> | <sentencia> <bloque> | <bloque>
<sentencia> ;
<sentencia>::= <sentencia_asignacion> | <llamada_a_funcion> |
<sentencia_if> |
<sentencia_for> | <sentencia_while> | <sentencia_switch>
}

-Errores:

- Le falta la definicion de los simbolos Terminales y los simbolos distinguidos de la gramatica, **T** y **S** respectivamente
- En la produccion de <bloque> tenes gramatica ambigua ya que tenes recursividad doble, tanto por deercha como por izquierda, lo

que provoca que la sentencia se pueda derivar de mas de una forma y eso no puede pasar

-En N se definen <otro>, <operacion>, <llamada_a_funcion>, <numero> pero en P no estan definidos

-En P se utilizan <sentencia_asignacion>, <sentencia_if>, <sentencia_while>, <sentencia_switch> y no estan definidos ni en P ni declarados en N

Ejercicio 12: Realice en EBNF la gramática para la definición un tag div en html 5. (Puede ayudarse con el siguiente enlace (<https://developer.mozilla.org/es/docs/Web/HTML/Elemento/div>))

```
G = (N,T,S,P)
```

```
N = {<div>, <palabra>, <tag>, <elem>, <tagCierre>, <letra>}
```

```
T = {a ... Z, A ... Z, <, >, /, , }
```

```
S = <div>
```

```
P = {
```

```
<div> ::= "<div>" { ( <elem> | <palabra> ) }+ "</div>"
```

```
<elem> ::= <tag> { ( <elem> | <palabra> ) }+ <tagCierre>
```

```
<tag> ::= "<" <palabra> ">"
```

```
<tagCierre> ::= "</" <palabra> ">"
```

```
<palabra> ::= {<letra>}+
```

```
<letra> ::= ( a | ... | z | A | ... | Z )
```

```
}
```

(Ejemplo de referencia):

```

<div>
  <p>
    Cualquier tipo de contenido aquí. Como <p>,
    <table>. ¡Lo que
    quieras!
  </p>
</div>

```

Ejercicio 13: Defina en EBNF una gramática para la construcción de números primos. ¿Qué debería agregar a la gramática para completar el ejercicio? → Ni idea

Ejercicio 14: Sobre un lenguaje de su preferencia escriba en EBNF la gramática para la definición de funciones o métodos o procedimientos (considere los parámetros en caso de ser necesario)

Tomando Java como lenguaje:

```

G = (N,T,S,P)

N = {<funcion>, <palabra> , <letra>, <digito>, <parametros>}

T = { a ... z, A ... Z , 0 .. 9 }

S = <funcion>

P = {

<funcion> ::= ("public" | "private") <palabra><palabra>
<parametros>
"{ " {<palabra>}+ "}"

<palabra> ::= <letra> { <letra> | <digito> }*

```

```
<letra> ::= ( a | ... | z | A | ... | Z )
```

```
<digito> ::= ( 0 | ... | 9 )
```

```
<parametros> ::= "(" [ <palabra> {"," {<palabra>}+ }  
")"
```

```
}
```